

repetitive mistakes over time, and AI helps mitigate these while enabling faster delivery of critical products.

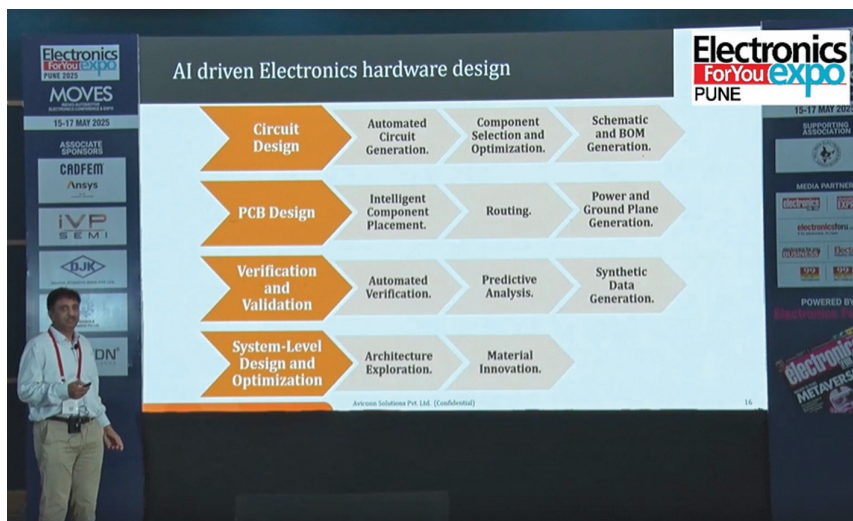
Although it is difficult to quantify the exact improvement compared to legacy processes, building a system manually can take months, whereas AI can generate code, documentation, and flowcharts almost instantly. In one test, a flowchart created manually for a commercial product was closely matched by an AI-generated version, which also included additional useful features.

For generating flowcharts, Python is commonly used with generative AI engines. Developers can use Python along with API libraries to generate both flowcharts and code.

In terms of trust, AI cannot fully replace human engineers, as human intelligence brings judgement and creativity that machines lack. AI can save time and handle repetitive tasks, but production-level output remains the responsibility of engineers.

Another important use of AI in embedded systems is integrating it as a component within the product itself. The flow is similar to any AI system, as follows:

- **Data acquisition.** When data is acquired by sensors from the environment or the system itself
- **Edge processing.** When data is processed on microcontrollers, FPGAs, or heterogeneous platforms to make quick inferences
- **Decision making.** Decisions are made based on processed data, and subsequent actions are determined
- **Actuation and control.** Decisions are converted to actions and the motors move, valves open, displays update, or any control mechanism responds
- **Connectivity and feedback.** Connectivity is essential for closing this loop and scaling AI-driven improvements. Data and results are sent to the cloud, enabling



Tapan Kapoor showing another slide

continuous learning. Insights from multiple devices refine the system over time, ensuring smarter and more efficient operation

Growth and challenges

This flow can be mapped to embedded systems to deliver tangible benefits. Recent trends are further optimising both products and embedded solutions. For instance, TinyML enables machine learning on computationally simple microcontrollers, such as AVR-based microcontrollers that are more capable than the ATmega328 commonly used in Arduino platforms.

Edge AI has three levels:

- **Full Edge AI.** Full-featured AI at the edge, handling larger workloads
- **Thin Edge AI.** Lightweight AI performing moderate processing near the edge
- **Micro Edge AI.** Positioned next to sensors, handling small data streams, making local decisions, and refining sensing in real time

The following are used to enhance their processing power:

- **Multi-core microcontrollers and SoCs** to support these operations
- **Neural processing units (NPUs)** are emerging, and neuromorphic computing is an early-stage concept that could eventually

complement or replace NPUs

AI Developer platforms now provide tools to:

- **Simulate and test** embedded systems, including virtual hardware and software partitioning, to identify bottlenecks and refine designs
- **Python APIs and frameworks** are increasingly available for embedded systems, simplifying integration

Developer tools and integration. AI developer platforms provide tools to simulate and test embedded systems, including virtual hardware, hardware–software partitioning, bottleneck identification, and design refinement. Python APIs and frameworks are increasingly available, simplifying integration into embedded systems.

Cybersecurity and secure design. Cybersecurity and secure design are now addressed more holistically. Early IoT devices often prioritised functionality over security, sometimes with serious consequences. Today, secure-by-design principles are applied from the outset, with early simulation ensuring compliance with standards and system reliability before deployment.

Real-time applications and opportunities

Energy efficiency. Embedding computationally-heavy AI in